

RTL Design Sherpa

GATLING-RV32ITETRAD Micro-Architecture Specification 0.1

October 22, 2026

Table of Contents

1 Gatling Mas Index..... 10

2 Document Information..... 11

2.1 GATLING-RV32ITETRAD Micro-Architecture Specification..... 11

2.2 Document Purpose..... 12

2.3 References..... 13

2.4 Terminology..... 15

2.5 Revision History..... 16

3 Overview..... 17

3.1 Machine organization..... 17

3.2 The machine in one picture..... 18

3.2.1 Figure 1.1: gatling datapath block diagram..... 18

3.3 What is reused..... 19

3.4 Owner decisions this build implements..... 20

4 gatling_core..... 21

4.1 Purpose..... 21

4.2 Interface..... 22

4.3 The stages..... 24

4.4 Per-context state..... 26

4.5 Halts (context-lock)..... 27

5 gatling_regfile..... 28

5.1 Purpose..... 28

5.1.1 Figure 2.1: gatling_regfile composition..... 28

5.2 Interface..... 29
5.3 Structure..... 30
6 gatling_mem_bram..... 31
6.1 Purpose..... 31
6.1.1 Figure 2.2: gatling_mem_bram composition..... 31
6.2 Memory..... 32
6.3 Loader rules..... 33
6.4 Register map..... 34
7 Internal Interfaces..... 35
7.1 The memory contract: one stage of latency, no wait..... 35
7.2 Run and halt between core and memory system..... 36
8 Micro-Architecture Behavior..... 37
8.1 The rotation, cycle by cycle..... 37
8.1.1 Figure 4.1: The rotation, cycle by cycle..... 37
8.2 The rotation on the pins..... 38
8.2.1 Waveform 4.1: One rotation window on the pins..... 38
8.3 Taken branches cost nothing..... 39
8.4 Stopping and restarting a context..... 40
9 Verification..... 41
9.1 DV (dv/tests, make/tests.mk rails)..... 41
9.2 Formal (formal/gatling/riscv-formal)..... 43
10 Configuration..... 44
11 Board Result..... 46
11.1 Nexys A7-100T (2026-10-21)..... 46

11.2 The worst path....47
11.3 Against kestrel....48
12 GATLING-RV32ITETRAD Hardware Architecture Specification Index....49
12.1 Document Organization....50
12.1.1 Front Matter....50
12.1.2 Chapter 1: Introduction....50
12.1.3 Chapter 2: Programmer's Model....50
12.1.4 Chapter 3: Interfaces....50
12.1.5 Chapter 4: Performance....50
12.2 Related Documentation....51
13 The Barrel Model: Why Four Contexts....52
13.1 1. The cost of one cycle....53
13.2 2. The obvious fix, and why it does not pay for one stream....54
13.3 3. The interleave....55
13.4 4. What the four stages buy in memory....56
13.5 5. What it costs....57
13.6 6. Measured....58
13.7 7. Sources....59
14 KESTREL-RV32I Micro-Architecture Specification Index....61
14.1 Document Organization....62
14.1.1 Front Matter....62
14.1.2 Chapter 1: Overview....62
14.1.3 Chapter 2: Functional Blocks....62
14.1.4 Chapter 3: Internal Interfaces....62

14.1.5 Chapter 4: Micro-Architecture Behavior....62
14.1.6 Chapter 5: Verification Hooks....62
14.1.7 Chapter 6: Configuration....62
14.2 Related Documentation....63
15 GATLING PRD — Tetrad Compute Engine....64
15.1 1. Purpose....65
15.2 2. Non-goals....66
15.3 3. ISA — RV32I + Xcell....67
15.3.1 3.1 Xcell encoding (custom-0, opcode 0x0B)....67
15.3.2 3.2 Assembly mnemonics....69
15.3.3 3.3 CSRs, and the deliberate departures from the RISC-V spec....69
15.4 4. Barrel microarchitecture....71
15.4.1 4.1 The model....71
15.4.2 4.2 Structural hazards (all resolved by rotation, by construction)....71
15.4.3 4.3 Branch behavior....71
15.4.4 4.4 Halt/trap....71
15.5 5. Contract fit (cell PRD 0.1, unchanged interfaces)....72
15.6 6. Verification....73
15.7 7. Open items (owner decisions needed)....74
15.8 8. Deliverables & sequencing....75
15.9 9. History....76
16 Microarchitecture Reading List....77
16.1 Foundations — kestrel & merlin....78
16.2 Prediction, traps, caches — peregrine....79

16.3 Out-of-order — gyrfalcon....80
16.4 ISA rationale & prior art — all rungs....81
16.5 Interleaved multithreading — gatling....82
16.6 Textbooks....83

List of Figures

Figure 1.1: gatling datapath block diagram.....18
Figure 2.1: gatling_regfile composition.....28
Figure 2.2: gatling_mem_bram composition.....31
Figure 4.1: The rotation, cycle by cycle.....37

List of Tables

Table 1: Reference Documents....13
Table 2: Terminology and Definitions....15
Table 3: Document Revision History....16
Table 4: Table 1.1: Hazards the rotation removes by construction....17
Table 5: Table 1.2: PRD decisions realized in the RTL....20
Table 6: Table 2.1: gatling_core ports....22
Table 7: Table 2.2: Work per stage....24
Table 8: Table 2.3: gatling_regfile ports....29
Table 9: Table 2.4: gatling_mem_bram registers (addr[17] = 1; memory below, bank in addr[15:14])....34
Table 10: Table 3.1: Request and data stages....35
Table 11: Table 5.1: DV....41
Table 12: Table 6.1: Configuration surface....44
Table 13: Table 7.1: gatling on the Nexys A7-100T....46
Table 14: Table 1.1: kestrel's single cycle on the Nexys A7....53
Table 15: Table 4.1: every memory latency is exactly one stage....56
Table 16: Table 6.1: The single cycle against the barrel....58
Table 17: Table 7.1: Sources cited in this chapter....59
Table 18: Textbooks cited by the rung books....83

List of Waveforms

Waveform 4.1: One rotation window on the pins.....38

1 Gatling Mas Index

Generated: 2026-10-22

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Document Information

2.1 GATLING-RV32ITETRAD Micro-Architecture Specification

Document Number: GATLING-MAS-001 **Version:** 0.1 **Status:** Draft **Classification:** Open Source - MIT License

2.2 Document Purpose

This Micro-Architecture Specification (MAS) describes how gatling, the four-context barrel RV32I, is built: the rotation that interleaves four hardware contexts through one F/D/X/W datapath, the context register files, the block-RAM memory system with one bank per context, per-context halts and run control, and how the design is verified. It is written for implementers and maintainers of the RTL and for reviewers checking it against the PRD.

The ISA and halt contract are kestrel's, and gatling uses kestrel's decode, immediate generator and ALU unchanged; their internals are in the KESTREL MAS and are not repeated here. The external contract is the companion GATLING HAS.

Companion Documents: - GATLING-RV32ITETRAD Hardware Architecture Specification (HAS) - The Barrel Model (`docs/barrel_model/barrel_model.md`) - why four contexts - KESTREL-RV32I MAS - the shared decode, immediate and ALU blocks

---

2.3 References

Reference Documents

ID	Document	Description
[REF-1]	GATLING PRD v0.3 (`PRD.md`)	Requirements and owner decisions
[REF-2]	KESTREL MAS	The decode / imm_gen / ALU blocks gatling reuses
[REF-3]	RISC-V Instruction Set Manual, Volume I (`projects/components/riscv-ip/references/riscv-spec.pdf`)	ISA authority
[REF-4]	riscv-formal `rvfi.md` (`vendor/riscv-formal/docs/rvfi.md`)	RVFI retire-channel semantics
[REF-5]	GATLING HAS (`docs/gatling_has/`)	The external contract this MAS implements (companion)
[REF-6]	The Barrel Model (`docs/barrel_model/barrel_model.md`)	Why four contexts, with the kestrel-against-gatling measurement
[REF-7]	RISC-V Instruction Set Manual, Volume II: Privileged Architecture (same PDF as [REF-3])	The CSRs the stub stands in for; cited as “priv §N”
[REF-8]	J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed., Morgan Kaufmann, 2019	Fine-grained multithreading, §3.11; pipeline hazards, Appendix C; the lineage from the CDC 6600 I/O processors through HEP, Tera and Niagara, Appendix M.5; cited as “H&P6 §N”
[REF-9]	D. A. Patterson and J. L. Hennessy, *Computer Organization and Design, RISC-V Edition*, 2nd ed., Morgan Kaufmann, 2021	Data and control hazards, §4.8-4.9; hardware multithreading, §6.4; cited as “P&H §N”
[REF-10]	S. L. Harris and D. M.	Multithreading as replicated

ID	Document	Description
	Harris, *Digital Design and Computer Architecture, RISC-V Edition*, Morgan Kaufmann, 2022	architectural state, §7.7.7; cited as "H&H §N"
[REF-11]	Microarchitecture reading list (`projects/components/riscv-ip/references/microarchitecture-papers.md`), entries 16-20	The barrel's primary sources: Thornton (CDC 6600), Smith and Jordan (HEP), Alverson et al. (Tera), Kongetira et al. (Niagara); cited as "(papers, entry N)"

2.4 Terminology

Terminology and Definitions

Term	Definition
Context (hart)	One of four hardware threads: a PC, a register file, a halt state and a run bit
Rotation	The counter (0..NUM_HARTS-1) that names the context in F; D, X and W hold the three before it
Window	Four cycles: the interval between two instructions of one context
Context-lock	A halt that stops only its own context
Beat	One retired instruction on the RVFI port (tagged with its context)

2.5 Revision History

Document Revision History

Version	Date	Author	Description
0.1	2026-10-22	seang	Initial MAS: the RV32I barrel as built (gatling_core, gatling_regfile, gatling_mem_bram); `NUM_HARTS` a parameter (default 4, at least the pipeline depth, capped at 16 by the memory system); the decode / immediate-generator / ALU blocks and ISA package shared from `riscv-ip/common-rv32i`; the CSR-stub cell; the figures (datapath, regfile and mem_bram block diagrams, the rotation grid, Waveform 4.1) and the references. Four harts on the Nexys A7. Xcell not yet built (custom-0 decodes as illegal).

Last Updated: 2026-10-22

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

3 Overview

3.1 Machine organization

gatling is one RV32I datapath shared by four hardware contexts. Each context owns a PC, a 32-entry register file, a halt state (halted, cause) and a run bit; everything else – decode, immediates, ALU, branch comparator, load/store alignment – exists once. That is the replicated architectural state of hardware multithreading as the textbooks define it: “it replicates only the PC and register file, not the execution units and memories” (H&H §7.7.7).

The contexts (`NUM_HARTS`, four by default: the tetrad) are interleaved through a four-stage pipeline in strict rotation. A counter, `sel_q`, names the context in F; D, X and W hold the contexts one, two and three before it. Every cycle each stage holds a different context, so a context has exactly one instruction in flight. That single fact removes every hazard a pipeline normally has to manage:

Table 1.1: Hazards the rotation removes by construction

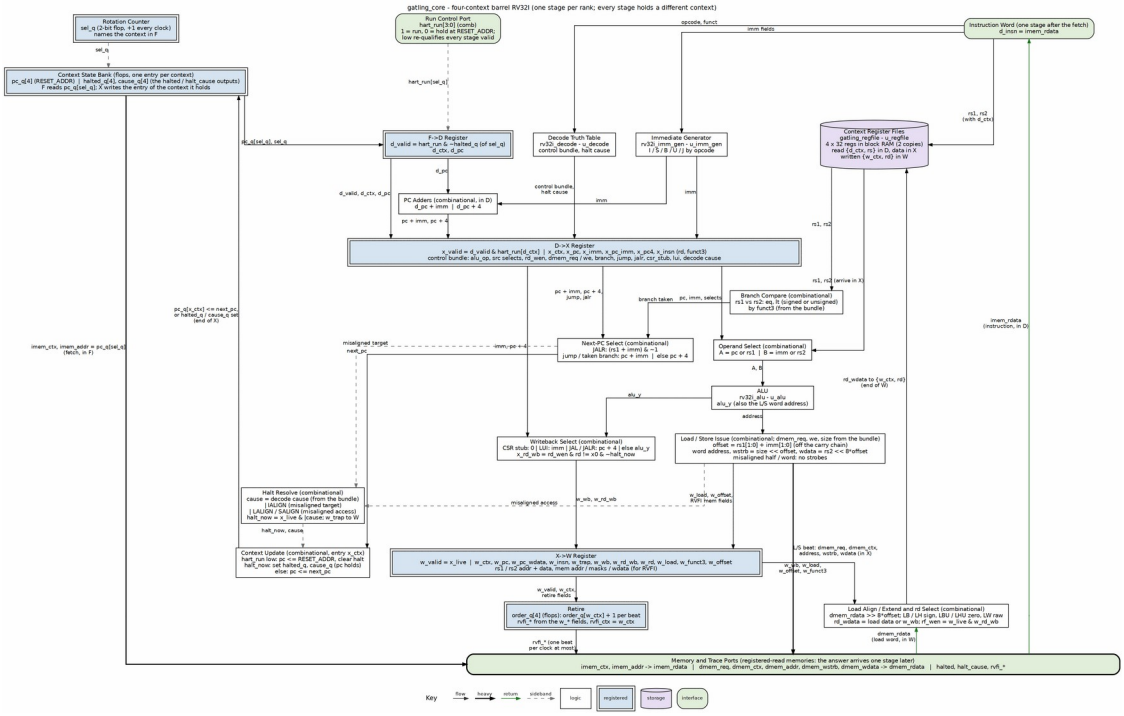
Hazard	Why it cannot happen
Data (RAW)	A context’s next instruction is in D at t+5; its previous instruction wrote its register at the end of W, t+3
Control	The branch target is written to the context’s PC at the end of X, t+2; the next fetch is at t+4
Structural	Each stage is used by one context per cycle; each memory port by one stage

These are the three hazard classes of the pipelining chapters (P&H §4.8-4.9; H&P6 App. C), and the textbook answer to each – forwarding, stalls, prediction and flushes – is machinery gatling does not contain. The rotation is fine-grained multithreading (H&P6 §3.11; P&H §6.4) with one departure: a held or halted context’s slot stays empty rather than being given to another context (chapter 4), because a context that issued early would have two instructions in flight and the hazards would return. The lineage, from the CDC 6600 peripheral processors through the HEP to Niagara, is in the barrel model [REF-6] and the reading list [REF-11].

Each context retires one instruction per window (CPI 4); with all four running the core retires one instruction per clock.

3.2 The machine in one picture

3.2.1 Figure 1.1: gatling datapath block diagram



gatling datapath block diagram

The figure is drawn from `gatling_core.sv`: the ranks are the four stages, the blue double-bordered rows between them are the stage registers, and every block names its RTL. Requests go down to the memory bar (the fetch address from the context bank on the left, the load/store beat from X in the center) and the answers climb back one stage later on their own lanes: the instruction word up the right edge into D, the load word into W. The two feedback climbs are the architectural writes: the context update into the context bank at the end of X, and the register write into the register files at the end of W. There are no hazard or forwarding lanes to draw, because each context has one instruction in flight.

3.3 What is reused

gatling's ISA contract is kestrel's (PRD 3): the same opcode map, the same trap/halt causes, the same CSR stub, FENCE/FENCE.I as NOPs, MRET falling through. The blocks that implement it are the `riscv-ip/common-rv32i` blocks kestrel also instantiates, used unchanged through their own filelists (`$RV32I_COMMON_ROOT`): `rv32i_decode`, `rv32i_imm_gen`, `rv32i_alu` and `rv32i_pkg`. One ISA has one source; a fix to the shared decode is a fix to both cores.

New for the barrel: `gatling_core` (the rotation, the per-context state and the stage registers), `gatling_regfile` (four register files in one block RAM) and `gatling_mem_bram` (one memory bank per context, the AXIL loader, per-context run and status).

3.4 Owner decisions this build implements

Table 1.2: PRD decisions realized in the RTL

Decision (PRD v0.3)	As built
Context visibility: implicit	No context-ID read; software cannot tell contexts apart
Program placement: separate	One 16 KB program/data bank per context, seen at every address
Illegal instruction: context-lock	Every halt stops only its context; the other three keep running
Scope: RV32I barrel first	Xcell (custom-0) decodes as illegal until the cell seam exists

4 gatling_core

4.1 Purpose

The barrel core: the rotation, one PC per context, per-context halt and run control, the stage registers between F, D, X and W, and the RVFI retire port. It instantiates the common-rv32i decode, immediate generator and ALU (shared with kestrel), and `gatling_regfile`.

4.2 Interface

Table 2.1: gatling_core ports

Port	Dir	Stage	Description
`clk`, `rst_n`	in		Clock; asynchronous-assert reset, synchronized internally (`reset_sync`, N=3)
`hart_run[NUM_HARTS-1:0]`	in	all	Per context: 1 = run, 0 = hold at RESET_ADDR (kills any of its instructions in flight)
`imem_ctx`, `imem_addr`	out	F	Fetch request: context and PC
`imem_rdata`	in	D	The fetched word, one cycle after the request (registered read)
`dmem_req`, `dmem_ctx`, `dmem_addr`, `dmem_wstrb`, `dmem_wdata`	out	X	Data access; `dmem_addr` is word-aligned, strobes and data lane-rotated
`dmem_rdata`	in	W	The load word, one cycle after the request
`halted[NUM_HARTS-1:0]`, `halt_cause[NUM_HARTS-1:0][3:0]`	out		Per-context halt state (rv32i_pkg causes)
`rvfi_*`, `rvfi_ctx`	out	W	One retirement per cycle at most, tagged with its

Port	Dir	Stage	Description
			context; `rvfi_order` counts that context's retirements

Parameters: `NUM_HARTS` (default 4, the tetrad; at least the pipeline depth of 4, checked at elaboration, and not necessarily a power of two since the rotation counter wraps explicitly – the context-index ports are `$clog2(NUM_HARTS)` wide) and `RESET_ADDR` (default 0; the board and the rv32ui battery use 0x8000_0000). `gatling_regfile` and `gatling_mem_bram` take the same `NUM_HARTS` and size their context arrays from it.

4.3 The stages

Figure 1.1 (Chapter 1) draws the stages and the lanes between them; the table names the work and the registers each stage leaves behind.

Table 2.2: Work per stage

Stage	Context	Work	Registers out
F	`sel_q`	PC of the context to the instruction RAM	`d_valid`, `d_ctx`, `d_pc`
D	`d_ctx`	Fetched word decoded (rv32i_decode) and its immediate built (rv32i_imm_gen); register read issued with `{d_ctx, rs1}`, `{d_ctx, rs2}`; PC+imm and PC+4 formed	control bundle, `x_pc_imm`, `x_pc4`, `x_imm`, `x_insn`
X	`x_ctx`	Operands arrive from the register file; ALU, branch comparator, next PC, misalignment and halt; data access issued; the context's PC (or halt) written	`w_*`: the writeback value, load selectors, RVFI fields
W	`w_ctx`	Load word arrives, aligned and extended; register write `{w_ctx, rd}`; RVFI beat	

PC+imm and PC+4 are formed in D because both operands are known there, which keeps the adders off X; X only selects.

4.4 Per-context state

`pc_q[c]`, `halted_q[c]`, `cause_q[c]` and `order_q[c]` are the only per-context state besides the register file. `hart_run[c]` low wins every update: the PC returns to RESET_ADDR and the halt and order clear. Otherwise the X stage updates the context it holds – its PC to the next PC, or, if the instruction halts, its halt bit and cause (the PC is not advanced, as kestrel).

Validity is carried per stage (`d_valid`, `x_valid`, `w_valid`) and re-qualified by `hart_run` of the stage's context at every stage, so stopping a context kills its in-flight instruction in whichever stage holds it: no store, no PC update, no register write, no retirement.

4.5 Halts (context-lock)

The X stage raises a halt for decode's causes (ecall, ebreak, illegal – including the not-yet-built Xcell custom-0 opcode), a misaligned control-flow target (IALIGN=32) and a misaligned load or store, exactly as kestrel. The halting instruction retires as an RVFI trap beat in W – the same cycle `halted[c]` rises – with no register write and no memory access. The context's next F slot sees `halted_q[c]` and fetches nothing; the other three contexts are unaffected.

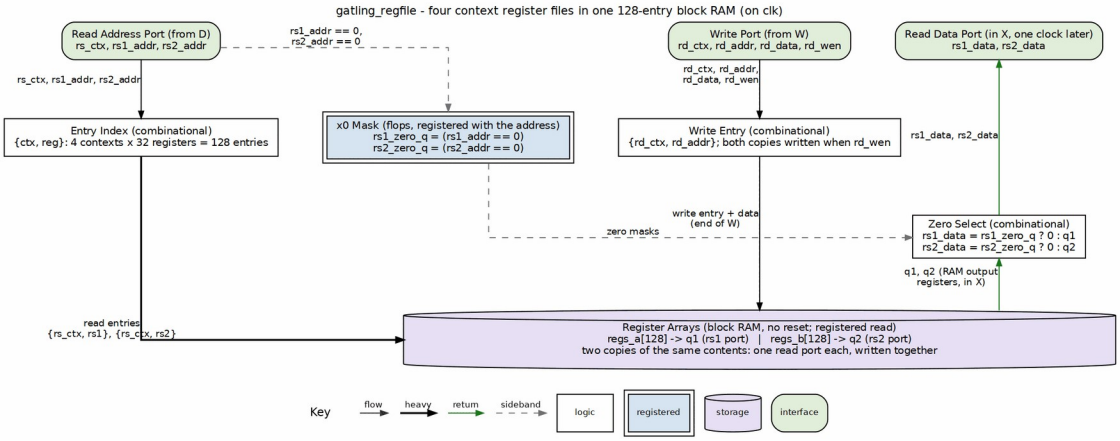
RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

5 gatling_regfile

5.1 Purpose

The NUM_HARTS context register files (four by default) as one NUM_HARTS x 32-entry array (128 by default) indexed {context, register}, with two registered read ports and one write port, in block RAM.

5.1.1 Figure 2.1: gatling_regfile composition



gatling_regfile: entry index, x0 mask, two block RAM copies, zero select

The read address enters at the top left in D, the write at the top center in W; the two block RAM copies are the storage bar, and the registered read data climbs its own lane through the x0 zero select to the read data port, in X.

5.2 Interface

Table 2.3: gatling_regfile ports

Port	Dir	Description
`rs_ctx`, `rs1_addr`, `rs2_addr`	in	Read address (D stage)
`rs1_data`, `rs2_data`	out	Read data, one cycle later (X stage)
`rd_ctx`, `rd_addr`, `rd_data`, `rd_wen`	in	Write (W stage)

5.3 Structure

Two copies of the array (`regs_a`, `regs_b`), each written on every write, give two read ports over one write port; each maps to one block RAM. The read is registered, which is the latency the barrel already has between D and X, so block RAM costs no cycle.

x0 reads zero through a read-side mask: `rs*_zero_q` registers whether the address was x0 and zeroes the RAM output. kestrel instead writes x0 to zero in reset; with four contexts that would be four reset writes, and the mask also makes a never-written RAM read zero for x0 in formal, where the array starts unconstrained.

The array has no reset (SRAM rule). D and W always hold different contexts, so the barrel never reads and writes the same entry in one cycle.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

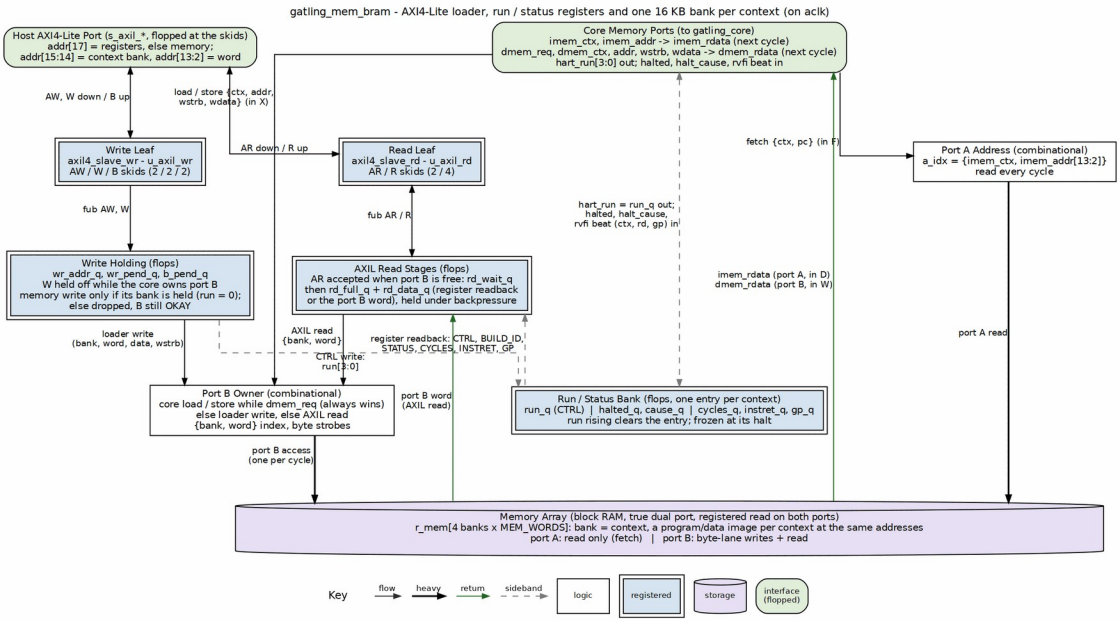
---

6 gatling_mem_bram

6.1 Purpose

The board's memory system: one program/data bank per context in one true-dual-port block RAM, the AXIL loader, per-context run control and per-context status registers. Derived from `kestrel_mem_bram`: the same AXIL leaves, the same load/run discipline and the same register semantics, widened to four contexts.

6.1.1 Figure 2.2: gatling_mem_bram composition



gatling_mem_bram: AXIL leaves, write holding, read stages, run/status bank, port B owner, four-bank block RAM

The host's AXI4-Lite port is the left column and the core's memory ports the right one; the four-bank true-dual-port array is the bar. Port A is the core's fetch port and nothing else. Port B is shared: the core's load/store when it has one (it always wins), else a loader write, else an AXIL read. Every read is registered, which is exactly the one stage the barrel allows. The run/status bank is sideband: it owns `hart_run` and collects each context's halt and retire events for the STATUS registers.

6.2 Memory

`MEM_WORDS` words per context (default 4096, 16 KB), indexed `{ctx, word}`. A context's bank is its whole address space – the bank index is the context, never an address bit – so four images linked at the same base (the riscv-tests link at 0x8000_0000) load side by side.

- Port A fetches: `{imem_ctx, imem_addr}` in F, the word in D.
- Port B is the core's data port while a context's load or store is in X (word in W), and the loader's otherwise. The core always wins; the loader waits a cycle (W is held off while the core owns port B).
- A store and a fetch never touch the same word on one edge: they belong to different contexts, so different banks.

6.3 Loader rules

A loader write lands only in a bank whose context is held (run = 0); a write to a running context's bank is acknowledged and dropped, as kestrel's loader drops writes while running. Reads are served any time port B is free. So one context can be reloaded and restarted while the other three run.

6.4 Register map

Table 2.4: gatling_mem_bram registers (addr[17] = 1; memory below, bank in addr[15:14])

Address	Name	Description
0x2_0000	CTRL	W: bits[NUM_HARTS-1:0] run per context. R: {halted, run}, NUM_HARTS bits each ({halted[7:4], run[3:0]} for four)
0x2_0004	BUILD_ID	ASCII “GATL” (0x4741_544C)
0x2_0100 + 0x10*c	STATUS	{cause[7:4], halted[1], run[0]} for context c
+0x4	CYCLES	Clocks since its run was set, frozen at its halt
+0x8	INSTRET	Its retired instructions (RVFI beats, the trap beat included)
+0xC	GP	The last value it wrote to x3: the riscv-tests verdict

Status holds its values after the context is stopped and clears when its run bit rises.

7 Internal Interfaces

7.1 The memory contract: one stage of latency, no wait

Every memory the barrel touches answers exactly one stage after it is asked:

Table 3.1: Request and data stages

Memory	Request	Data	Provided by
Instruction	F	D	`gatling_mem_bram` port A
Register file	D	X	`gatling_regfile`
Data (load)	X	W	`gatling_mem_bram` port B

This is why gatling has no `mem_wait`. kestrel, single-cycle, has to stall a cycle on every block-RAM load; the barrel's stages are exactly the block RAM's latency, so nothing waits. A memory system for gatling must keep this contract on both ports, which `gatling_mem_bram` does by registering both reads and by giving the core port B unconditionally.

7.2 Run and halt between core and memory system

`gatling_mem_bram` drives `hart_run` from CTRL; the core reports `halted` and `halt_cause` per context, and the memory system latches them into STATUS. The core's own halt state clears while a context is held; the memory system's STATUS keeps the verdict until the next run, so the host can read it after stopping the context.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

8 Micro-Architecture Behavior

8.1 The rotation, cycle by cycle

8.1.1 Figure 4.1: The rotation, cycle by cycle

stage \ cycle	t	t+1	t+2	t+3	t+4	t+5	t+6	t+7
F fetch at pc_q[sel_q]	c0 (i)	c1	c2	c3	c0 (i+1)	c1	c2	c3
D decode, register read issued	-	c0 (i)	c1	c2	c3	c0 (i+1)	c1	c2
X execute; PC written at the end	-	-	c0 (i)	c1	c2	c3	c0 (i+1)	c1
W retire; register written at the end	-	-	-	c0 (i)	c1	c2	c3	c0 (i+1)

Shaded: context 0. Its instruction i writes the PC at the end of t+2 and its register at the end of t+3;
instruction i+1 is fetched at t+4 and reads its registers at t+5 -- two cycles after each write, so no hazard exists.
Every cycle each stage holds a different context; a held or halted context leaves its cells empty and the others keep their slots.

The rotation as a cycle grid: one row per stage, one column per clock, context 0 shaded

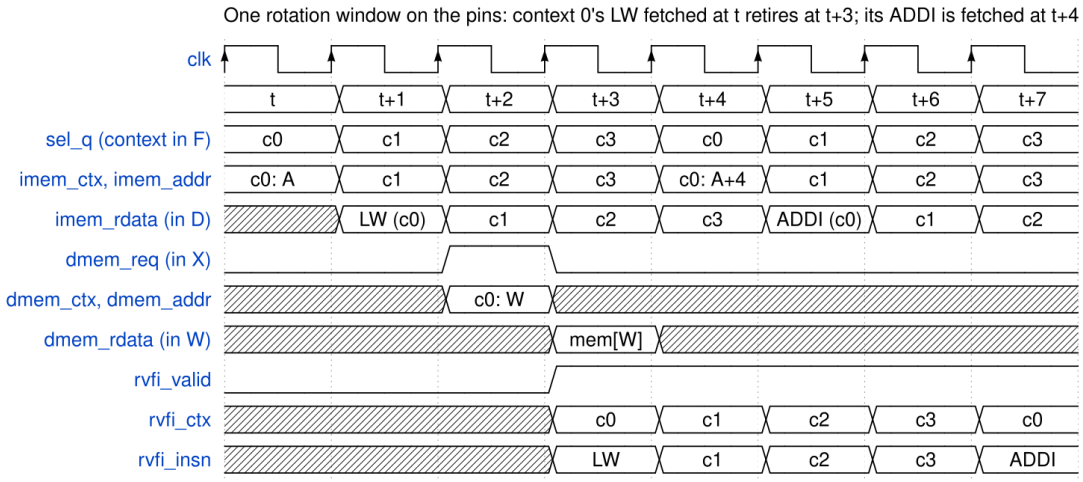
Context c0's instruction i is fetched at t, decoded at t+1, executed at t+2 (its PC written at the end of t+2) and retired at t+3 (its register written at the end of t+3). Instruction i+1 is fetched at t+4 from the PC written two cycles earlier, and reads its registers at t+5, two cycles after the write. Consecutive retirements of one context are exactly four cycles apart; with all four running, retirements follow the rotation order, one per clock.

A context that is held or halted leaves its slot empty: the other contexts are not shifted into it, so each still retires once per window. This is measured: in DV every context's consecutive beats are checked to be four cycles apart, and on gatling_mem_bram a context's CYCLES/INSTRET is 4.000.

The empty slot is the one place gatling departs from fine-grained multithreading as the textbooks describe it, where round-robin issue "skips any threads that are stalled at that time" (H&P6 §3.11; P&H §6.4) and Niagara does exactly that (papers, entry 20). Skipping would let the next instruction of a running context enter the pipeline fewer than four cycles behind its last, and the hazard logic the barrel exists to remove would come back (PRD 3.3). The HEP had the same rule in a different form: a process could issue only once its previous instruction had left the pipeline (papers, entry 17).

8.2 The rotation on the pins

8.2.1 Waveform 4.1: One rotation window on the pins



Waveform 4.1: one rotation window on the pins, context 0's load fetched at t and retired at t+3

The same window as the grid, on `gatling_core`'s ports. `sel_q` walks c0, c1, c2, c3 and the fetch port follows it; the fetched word is on `imem_rdata` one cycle after its address, which is the D stage of the context that asked for it. Context 0's LW issues its data access at t+2 (`dmem_req`, tagged `dmem_ctx = c0`), the word arrives at t+3 and the beat retires that cycle with `rvfi_ctx = c0`. From t+3 on, one beat retires every clock, in rotation order; context 0's next instruction, the ADDI at A+4, is fetched at t+4 from the PC written at the end of t+2.

8.3 Taken branches cost nothing

The branch decision and target are known in X and written to the context's PC at the end of X, two cycles before that context's next fetch. There is no wrong-path fetch to cancel, so no bubble, no flush and no prediction.

8.4 Stopping and restarting a context

`hart_run[c]` low kills context c's instruction in whichever stage holds it – the stage valid is re-qualified by `hart_run` everywhere – and holds its PC at RESET_ADDR. Released, the context fetches RESET_ADDR in its next F slot with its retirement order at 0. DV stops a context at all four rotation phases (one per stage) and checks that nothing retires while it is held; a mutant that drops the qualification in W retires one leaked beat and is caught only by the stop in the W phase.

9 Verification

9.1 DV (dv/tests, make/tests.mk rails)

Table 5.1: DV

Test	What it checks
`test_gatling_regfile`	Registered read on both ports, x0 zero in every context, contexts isolated, the barrel's read-D/write-W pattern; random traffic against a model at func/full
`test_gatling_core`	Four different programs interleaved, each golden-identical; every halt kind on its own context (context-lock); Xcell custom-0 locks only its context; run control (lone context, late start, stops at all four rotation phases, restart); the CSR stub (cycle, time, instret, their high halves, mcycle and mhartid read 0, writes dropped, nothing halts)
`test_gatling_core_num_harts`	The interleave, context-lock and CSR-stub cases again on an eight-context build (`NUM_HARTS=8`): the TB reads the hart count off the DUT and scales the programs, the window and the rotation check to it, so the parameter is exercised, not just elaborated. The register file is 256 entries there and the rotation counter wraps at 7
`test_gatling_rv32ui`	The riscv-tests rv32ui-p battery: four different images at once, rotated so every context runs all 42 (168 runs); golden full-field diff at every level, spike lockstep at func
`test_gatling_fuzz`	kestrel's constrained-random generator; four different streams at once; golden and spike on every

Test	What it checks
	stream at every level (5 / 25 / 200 streams)
`test_gatling_mem_bram`	Through AXIL: the map, banks, strobes, alias; a running bank refuses writes; reload one context while another runs; STATUS/CPI; the battery loaded and run entirely over AXIL

Each context's trace is checked with kestrel's own trace checks and against kestrel's golden interpreter and spike: gatling's ISA contract is kestrel's, so its goldens are too.

Mutation (each planted bug must turn a green suite red): register writes ignoring the context fail every directed test and the battery; the W stage ignoring `hart_run` fails run control. Both mutants first exposed weak checks – the gate battery checked only the verdict, and run control allowed one leaked beat – which were tightened before the RTL was trusted.

9.2 Formal (formal/gatling/riscv-formal)

riscv-formal observes one hart. The wrapper reports context 0's retirements and leaves contexts 1-3 free – their instructions, load data and run bits are unconstrained every cycle – with context 0 held running. All 42 checks pass. Beyond the RV32I insn checks on context 0, `reg`, `pc_fwd` and `pc_bwd` therefore prove that nothing the other contexts execute, and no way they are started or stopped, can disturb context 0's registers or PC. The rotation treats all four contexts alike, so the proof stands for each.

Depths are on context 0's retirement phase (cycle 3 mod 4): an insn check fires at one cycle and assumes a retirement there, and off-phase that assumption is unsatisfiable (sby PREUNSAT), not a vacuous pass. Non-vacuity was measured: an ADD bug fails insn_add at depth 11; register writes ignoring the context fail `reg`; context 0's PC written by other contexts fails `pc_fwd` and `liveness`.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

10 Configuration

Table 6.1: Configuration surface

Item	Where	Value
RESET_ADDR	gatling_core parameter	0 (board and battery: 0x8000_0000)
MEM_WORDS	gatling_mem_bram parameter	4096 per context (16 KB)
NUM_HARTS	gatling_core, gatling_regfile, gatling_mem_bram parameter	4 (the tetrad). At least the pipeline depth (4): fewer contexts than stages would put two instructions of one context in flight. More is legal (each context then owns 1/NUM_HARTS of the datapath); the rotation counter wraps explicitly, so a power of two is not required. gatling_mem_bram caps it at 16 (its per-context register block) and checks that the bank field fits below the register address bit. Both checks are elaboration-time $errors

Filelists (`rtl/filelists/`): `fub/gatling_regfile.f`, `fub/gatling_mem_bram.f`, `top/gatling_core.f` (pulls the common-rv32i decode/imm_gen/ALU filelists, which bring `rv32i_pkg` with them), `gatling_all.f`. The root variable is `GATLING_ROOT`; `RV32I_COMMON_ROOT` must also be set. There is no gatling package: the barrel adds no ISA vocabulary of its own.

11 Board Result

11.1 Nexys A7-100T (2026-10-21)

Build: `projects/fpga-systems/NexysA7/riscv-ip/gatling/build-bram` (gatling_core + gatling_mem_bram + the UART-to-AXI4-Lite bridge), IMPL=explore, from `make clean-all`. Host: `make host-gatling`, the same programs as the UART-equivalence cosim.

Table 7.1: gatling on the Nexys A7-100T

Measure	Value
Clock	100 MHz, worst slack +0.152 ns
rv32ui battery	168/168: 42 images x 4 harts, four different images at once per round
CPI per hart	4.004 / 4.005 / 4.004 / 4.004
Battery throughput	0.688 IPC (68.8 MIPS): rounds wait for their longest image
Peak throughput	1 IPC, 100 MIPS (four harts of 25)
LUTs / registers	2,281 / 1,742 (whole board top)
gatling_core	1,066 LUTs, 2 RAMB18 (the register files)
gatling_mem_bram	799 LUTs, 16 block RAM tiles (4 x 16 KB)
DSPs	0

11.2 The worst path

9.4 ns, 10 logic levels: register-file block RAM output -> branch comparator / next-PC and halt logic -> the clock enable of a context's PC register. This is the X stage; F, D and W are each a fraction of it. kestrel's worst path on the same board was 11.6 ns at IMPL=explore and covered the whole instruction.

11.3 Against kestrel

kestrel runs one program at 75 MHz and CPI 1.039: 72.2 MIPS for 1,899 LUTs. gatling runs four at 100 MHz and CPI 4 each: 100 MIPS peak for 2,281 LUTs – four harts for 20% more logic, and no wait states, hazards or branch penalty to manage. A single program on gatling runs at 25 MIPS, a third of kestrel's: the barrel pays off when there are four things to do (Barrel Model, section 5).

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

12 GATLING-RV32ITETRAD Hardware Architecture Specification Index

Version: 0.1 **Date:** 2026-10-22 **Purpose:** External architecture of gatling, the four-context barrel RV32I: what an integrator and a programmer see

---

12.1 Document Organization

Note: All chapters linked below for automated document generation.

12.1.1 Front Matter

- Document Information

12.1.2 Chapter 1: Introduction

- What gatling Is

12.1.3 Chapter 2: Programmer's Model

- Contexts, ISA and Halts

12.1.4 Chapter 3: Interfaces

- Core Ports and the Board Memory Map

12.1.5 Chapter 4: Performance

- Throughput and Clock

12.2 Related Documentation

- **GATLING MAS** - Micro-architecture
- **The Barrel Model** - Why four contexts
- **GATLING PRD** - Requirements, the Xcell encoding table, owner decisions
- **Reading list** - The barrel's primary sources (entries 16-20) and the textbook chapters cited

---

Last Updated: 2026-10-22 **Maintained By:** RTL Design Sherpa Project

13 The Barrel Model: Why Four Contexts

Version: 0.1 **Date:** 2026-10-22 **Purpose:** The study chapter behind gatling (PRD 8.1): what kestrel's single cycle costs, why cutting it into stages does not help one instruction stream, and how four interleaved streams make the cut free.

13.1 1. The cost of one cycle

kestrel executes an RV32I instruction in one clock: the program counter, the register-file read, the ALU, the data memory and the register write are one combinational path. That is the teaching point of kestrel and it is also its price. On the Nexys A7 the path that bounds the clock runs from the instruction block RAM, through the register read and the ALU add, into the data memory's address (kestrel MAS, "Board result"):

Table 1.1: kestrel's single cycle on the Nexys A7

kestrel, Nexys A7-100T	Value
Worst path at 100 MHz (IMPL=explore)	11.6 ns
Clock closed	75 MHz (+0.156 ns)
CPI	1.039 (1.00 plus a cycle per load)
Throughput	72.2 MIPS
LUTs	1,899

Every instruction pays for the longest one. A shift, an add and a load all wait the 11.6 ns that a load-address calculation needs.

13.2 2. The obvious fix, and why it does not pay for one stream

Cut the path into stages and clock it faster. Four stages – fetch, decode, execute, write back – each hold a quarter of the work, so the clock could rise toward four times kestrel's.

For one instruction stream the gain mostly evaporates. The next instruction depends on this one: its operands may be this instruction's result (a data hazard), and its address may be this instruction's branch target (a control hazard). A pipelined single-stream core needs forwarding paths, hazard detection, stall logic and branch prediction or flushes to keep the stages busy (P&H §4.8-4.9; H&P6 App. C) – the falcon-suite ladder above kestrel exists to teach exactly that. Without them, the simplest correct multi-cycle machine runs one instruction at a time through all four stages: a CPI of 4 at up to four times the clock, which is roughly where it started.

13.3 3. The interleave

gatling keeps the four stages and removes the dependence instead: the stages are fed from FOUR independent instruction streams, one per hardware context, taken in strict rotation.

This is fine-grained multithreading in the textbook sense – switch threads every cycle, in round-robin order (H&P6 §3.11; P&H §6.4; H&H §7.7.7) – and it is older than the pipelining vocabulary used to describe it. Thornton's CDC 6600 ran ten peripheral processors through one shared execution unit this way in 1964 (papers, entry 16; H&P6 App. M.5); the Denelcor HEP used it to hide pipeline and memory latency with no forwarding and no caches (papers, entries 17 and 18); the Tera MTA pushed it to 128 streams (papers, entry 19); Sun's Niagara brought it back as a four-thread core (papers, entry 20). The textbooks present the interleave as a way to *hide* stalls. gatling uses it to *remove* hazards, which is what happens when the number of streams is fixed equal to the number of stages:

```
cycle     t     t+1   t+2   t+3   t+4   t+5
F        c0    c1    c2    c3    c0    c1
D              c0    c1    c2    c3    c0
X                    c0    c1    c2    c3
W                          c0    c1    c2
```

Every cycle each stage holds a different context. Context c0's next instruction is fetched at t+4, after its current one has written its PC (end of X, t+2) and its register (end of W, t+3). So:

- no data hazard – a context never reads a register that an instruction of its own still in flight is about to write;
- no control hazard – a branch resolves before that context's next fetch, so there is nothing to predict and nothing to flush;
- no forwarding, no stalls, no interlocks for the base ISA.

Each context still takes four cycles per instruction (CPI 4), but four of them retire one instruction each per four cycles: the core retires one instruction per clock. The multi-cycle cut costs nothing when there are four streams to fill it – the kestrel to merlin lesson inverted: instead of pipeline stages hiding latency for one stream, contexts hide it for each other.

13.4 4. What the four stages buy in memory

Each stage is one block-RAM latency long, and that is not a coincidence the design has to work around; it is what the design uses:

Table 4.1: every memory latency is exactly one stage

Memory	Address presented	Data used
Instruction RAM	F	D
Register file (block RAM)	D	X
Data RAM (load)	X	W

kestrel has to hold the core for a cycle on every load (`mem_wait`) because its block RAM answers a cycle late. gatling has no wait signal at all.

13.5 5. What it costs

- Four PCs and four register files instead of one – the “cheap replication” (PRD 4.1). Harris and Harris make the same point: multithreading “replicates only the PC and register file, not the execution units and memories” (H&H §7.7.7). The register files are one 128-entry block RAM (two copies for two read ports), not 4,096 flops.
- One datapath: decode, immediates and ALU are kestrel’s, unchanged.
- A context is a quarter of the machine. A single program runs at a quarter of the clock rate – the fine-grained trade as Hennessy and Patterson state it, multithreaded throughput bought with single-thread latency (H&P6 §3.11), and what Jordan measured on the HEP (papers, entry 18). The barrel pays off only when there are four things to do – which, for a tetrad of four cells, is the point.

13.6 6. Measured

On the same Nexys A7-100T (gatling board README, “Board result”; MAS chapter 7):

Table 6.1: The single cycle against the barrel

	kestrel	gatling
Worst path	11.6 ns (the whole instruction)	9.4 ns (the X stage)
Clock	75 MHz	100 MHz
Programs at once	1	4
CPI per program	1.039	4.004
Peak throughput	72.2 MIPS	100 MIPS
LUTs	1,899	2,281

Four harts cost 20% more logic and run 39% more instructions per second in total, with every instruction taking the same four clocks. The clock did not rise fourfold: the X stage – register read through ALU or branch to the next PC – is still most of kestrel’s instruction, and routing is half of the worst path (52%) on this part. The remaining stages have slack to give, which is where a deeper rotation would go, as the HEP’s eight-stage pipeline and Tera’s 128 streams did (papers, entries 17 and 19).

13.7 7. Sources

Citations follow the suite's conventions (kestrel study book, chapter 1, "Conventions"): the ISA by section of the pinned manual, papers by entry in `references/microarchitecture-papers.md`, textbooks by edition and section.

Table 7.1: Sources cited in this chapter

Key	Source
H&P6	J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed., Morgan Kaufmann, 2019. §3.11 "Multithreading: Exploiting Thread-Level Parallelism to Improve Uniprocessor Throughput"; Appendix C "Pipelining: Basic and Intermediate Concepts"; Appendix M.5 "The Development of Pipelining and Instruction-Level Parallelism"
P&H	D. A. Patterson and J. L. Hennessy, *Computer Organization and Design, RISC-V Edition*, 2nd ed., Morgan Kaufmann, 2021. §4.8 "Data Hazards: Forwarding versus Stalling"; §4.9 "Control Hazards"; §6.4 "Hardware Multithreading"
H&H	S. L. Harris and D. M. Harris, *Digital Design and Computer Architecture, RISC-V Edition*, Morgan Kaufmann, 2022. §7.7.7 "Multithreading"
papers, entries 16-20	Thornton 1964 (CDC 6600); Smith 1978 and Jordan 1983 (HEP); Alverson et al. 1992 (Tera MTA); Kongetira et al. 2005 (Niagara)
kestrel MAS	Chapter 7, "Board result": the kestrel column of Tables 1.1 and 6.1

14 KESTREL-RV32I Micro-Architecture Specification Index

Version: 0.21 **Date:** 2026-10-22 **Purpose:** Block-level micro-architecture specification for the kestrel-rv32i single-cycle RV32I core

---

14.1 Document Organization

Note: All chapters linked below for automated document generation.

14.1.1 Front Matter

- Document Information

14.1.2 Chapter 1: Overview

- Design Philosophy and Machine Organization

14.1.3 Chapter 2: Functional Blocks

- kestrel_core (top)
- rv32i_decode
- rv32i_imm_gen
- rv32i_alu
- kestrel_regfile
- kestrel_mem_loader
- kestrel_mem_bram

14.1.4 Chapter 3: Internal Interfaces

- Package Types and Top-Level Wiring

14.1.5 Chapter 4: Micro-Architecture Behavior

- The Single Cycle, Step by Step
- Misaligned Data Access Halt
- Halt, IALIGN, and the System Layer
- RVFI Aggregation

14.1.6 Chapter 5: Verification Hooks

- Verification Hooks and Observability

14.1.7 Chapter 6: Configuration

- Parameters, Defines, and Filelists

14.2 Related Documentation

- **KESTREL HAS** - Hardware Architecture Specification (external view)
- **Simplified RV32I: kestrel** - Tutorial-study book this core is documented by
- **RISC-V Falcon Suite README** - Suite overview and rung ladder
- **riscv-formal rvfi.md** - Retire-channel field semantics
- **Testplans** - V&V cross-reference (six plans)

Last Updated: 2026-10-20 **Maintained By:** RTL Design Sherpa Project

15 GATLING PRD — Tetrad Compute Engine

Version: 0.4 (2026-10-21 — the deliberate departures from the RISC-V spec recorded in 3.3; no Zicntr.) **Date:** 2026-10-21 **Owner decision inputs:** “Instructions” (over MMIO) for fabric handshake; mini RV per tetrad (over per-cell ALU+FSM); codename gatling (barrel machine). **Supersedes:** PRD 0.1 wording that described gatling as replacing the per-cell ALU — the ALU stays; gatling replaces the execution FSM as its driver.

15.1 1. Purpose

Give each tetrad of the compute tile a programmable engine small enough to instantiate four times per tile, powerful enough to run compiled programs over cell input data, and honest enough to teach: the barrel processor as the kestrel→merlin lesson, inverted — instead of pipeline stages hiding latency for one stream, N contexts hide the multi-cycle latency of one shared datapath.

Owner ruling 2026-10-19: gatling sequences the four ALUs of its adjacent cells. The barrel RV32I core is the programmable sequencer for its tetrad — each context runs a small program whose POP presents operands to the cell's alu-infer issue handshake and whose PUSH retires a result line; the ALU keeps its elastic valid/ready seam and tag-alignment contract unchanged. *The program is the config*. Mode bits that remain (element width, credit batch N) move to the CFG read path. Gatling is also a barrel RISC-V with custom commands (Xcell: CFG/POP/PUSH/ECTX/FLUSH/KICK/WAIT) — the program is the schedule. Gatling never originates config traffic; cell configuration arrives in-band over delta.

15.2 2. Non-goals

- Not a falcon-suite rung. No performance chase; IPC-per-context ~ 1 is the target, not a benchmark result.
- Not per-cell processors. One engine per tetrad; the four cells are the four contexts.
- No caches, no virtual memory, no interrupts/CLINT, no M/A/F. (Machine state is 4× {PC, x1..x31} + mode CSR shadow. Full stop.)
- No CSRs as a subsystem — `Zicsr` deliberately absent; config reaches the program through the Xcell `CFG` instruction. Every CSR access is kestrel's stub (3.3), the Zicntr counters included.

15.3 3. ISA — RV32I + Xcell

Base RV32I exactly as kestrel implements it (same opcode map, same trap/halt contract) so the spike lockstep flow and rv32ui battery carry unchanged. The Xcell extension occupies the custom-0 opcode (`0001011`, 0x0B) — reserved space, zero collision risk with future standard extensions.

15.3.1 3.1 Xcell encoding (custom-0, opcode 0x0B)

```
 31                      15 14   12 11      7 6       0
+-------------------------+-------+---------+--------+
|        imm[31:12]       | funct3|   rd    | 0x0B   |  CFG
|        000000000000     |  000  |   rd    | 0x0B   |  POP
|        000000000000     |  001  |  rs1    | 0x0B   |  PUSH
|        000000000000     |  010  |  00000  | 0x0B   |  ECTX
|        000000000000     |  011  |  rs1    | 0x0B   |  FLUSH
|        000000000000     |  100  |  rs1    | 0x0B   |  KICK
|        000000000000     |  101  |  rs1    | 0x0B   |  WAIT
+-------------------------+-------+---------+--------+
funct3 110/111 : reserved.
funct3 010 (ECTX) and 011/100/101 (FLUSH/KICK/WAIT) added in v0.2.
The v0.1 WAITCD/WAITGT placeholders are superseded by KICK/WAIT
pending
the tetrad MAS.
```

Instr	Action	Blocking behavior
`CFG rd, imm`	rd ← mode-register[imm]: 0=element mode (0=bf16: 2 bf16 lanes per 32-bit line, fp32 accumulate; other values reserved for the int8/fp32 roadmap per alu-infer PRD `func_sel[8:7]`), 1=credit batch N, 2=context ID (read-only, see §7 open item), 3+=reserved	never blocks
`POP rd`	rd ← next element of this context's input memory; head advances; consumed-line credit returned to rapids per credit-batch config	context stalls if input empty — hardware interlock, not a spin

Instr	Action	Blocking behavior
`PUSH rs1`	rs1 → this context's result memory; tail advances; grant requested on result path; packetizer batches X flits	context stalls if no grant
`ECTX`	context done: freeze PC, raise done status to tetrad MonBus, release datapath share; context restarts on reset/config reload	drains at next context boundary
`FLUSH rs1`	rs1[14:0] → cell `FLUSH_DEST` {ext,level,X,Y,Z}; rs1[18:15] → cell `FLUSH_TAG` (RAPIDS sink-channel tag); trigger cell flush-and-send of result memory	context stalls until drain controller accepts command
`KICK rs1`	raise tetrad coordination event on the tetrad MonBus with value `rs1`; provisional encoding pending tetrad MAS	non-blocking once event is posted
`WAIT rs1`	stall context until the tetrad coordination condition in `rs1` is observed (e.g. group-done bitmask); provisional encoding pending tetrad MAS	context stalls until condition true

Design note (owner doctrine): fixed hardware owns the line-rate handshake (credit/grant detection, memory ports); the program owns policy. POP/PUSH blocking interlocks subsume most waiting — a context that would wait simply isn't scheduled. WAIT is an explicit escape hatch for cross-context coordination, not the common case.

Element semantics: with element width < 32, POP zero-extends into rd and PUSH takes the low bits of rs1; packing/unpacking is the program's business (one ANDI/SLTI pair, honest and visible).

15.3.2 3.2 Assembly mnemonics

`gatling_pop`, `gatling_push`, `gatling_cfg`, `gatling_ectx`, `gatling_flush`, `gatling_kick`, `gatling_wait` as canonical toolchain names; assembler aliases `pop/push/cfg/ectx/flush/kick/wait` live in the Xcell binutils fragment when the flow arrives (§8).

15.3.3 3.3 CSRs, and the deliberate departures from the RISC-V spec

There is no CSR state. Zicntr (`cycle`, `time`, `instret`) is not implemented: the counters were built and then removed (owner 2026-10-21) – no gatling program reads them, and they cost about 344 LUTs per core. A program that needs a time base reads one from its memory map.

Deliberate departures (owner 2026-10-21, after a review against the RISC-V books in the library – Patterson & Hennessy, Harris & Harris, Ledin, Zhang):

Departure	The spec	Why gatling differs
`mhartid` reads 0 on every context	hart IDs are unique	contexts are implicit (7, item 2): no program can tell which cell it runs
Every CSR reads 0 and drops writes, the counters included	an unsupported CSR access is an illegal instruction	kestrel's stub, kept so the rv32ui p-env preamble (`csrw mtvec`, …) runs
No trap vector: a trap halts the context (4.4); MRET is a no-op	M-mode with `mtvec`/`mepc`/`mcause` is mandatory	context-lock is the halt contract; a tetrad is not a CPU
`WFI` is illegal and halts the context	`WFI` may be a no-op	waiting belongs to the Xcell `WAIT` (TASK-002), which has something to wait on
The rotation never skips a halted or held context	fine-grained multithreading usually skips stalled threads (H&P 3.11, P&H 6.4)	skipping would let a context's next instruction issue sooner than four cycles behind its last and bring back the hazard logic the barrel exists to remove; a tetrad runs all four

Departure	The spec	Why gatling differs
		cells

15.4 4. Barrel microarchitecture

15.4.1 4.1 The model

Kestrel's single cycle is PC + regfile-read + ALU + memory in one combinational path — the teaching point and the timing/area cost. Gatling splits that into a 4-state FSM:

```
F  : address = ctx[sel].pc; fetch word from ctx program image
D  : decode; read ctx[sel] regfile (rs1/rs2 by decoded index)
X  : ALU / Xcell unit (POP/PUSH/CFG execute here; interlock check)
W  : writeback to ctx[sel] regfile; pc advance (or branch target)
```

Each FSM state advances context `sel` one step; `sel` rotates 0→1→2→3→0. Steady state: every context retires one instruction per 4-cycle window ⇒ 1 IPC per context, one gatling ALU, one decoder, one Xcell unit, **four** register files + PCs (the cheap replication) — replacing the cell PRD's four execution FSMs while each cell keeps its own bf16 ALU datapath (owner ruling 2026-10-19).

15.4.2 4.2 Structural hazards (all resolved by rotation, by construction)

- ALU/decoder/memory-port conflict: impossible — one user per state.
- Input/result memory port conflict across contexts: impossible — each context owns its cell's memories, addressed by `sel`.
- Interlock stall (POP empty / PUSH no-grant): only `sel` stalls; rotation skips it (bubble) — three contexts keep the datapath busy.

15.4.3 4.3 Branch behavior

Resolved in X (kestrel semantics carry: the branch decision is one comparison) and written to the context's PC at the end of X, two cycles before that context's next fetch. As built (2026-10-21) a taken branch therefore costs nothing – no bubble, no flush machinery, no BTB, no prediction. The predictor study belongs to peregrine, not here.

15.4.4 4.4 Halt/trap

No traps (no exceptions in base scope; illegal instruction → context-local lock with status bit via MonBus, other contexts continue — a tetrad is not a CPU, one bad cell must not sink the tile). `ECTX` is the normal exit. Reset is per-tile from the tile root; program load via the fabric config path (PKT_CONFIG) into each cell's program image region — kestrel's `mem_loader`/AXIL pattern becomes the in-band loader, per the standing in-band-only fabric decision.

15.5 5. Contract fit (cell PRD 0.1, unchanged interfaces)

Cell PRD block	Stand / change
delta_mesh_cell node, containment	stands — gatling sits behind the node's local port
Input memory, line fill by node from rapids	stands; per-context, addressed by `sel`
Result memory, X-flit batching, packetizer, grants	stands
Consumed-line credits to rapids, batch N (default 1)	stands; N read via CFG
Element widths B4/8/16/32	superseded — alu-infer's bf16 element model (2 bf16 lanes per 32-bit line, fp32 accumulate) is normative; CFG[0] reports the alu-infer element mode (bf16 today; int8/fp32 modes are a roadmap item in the alu-infer PRD via `func_sel[8:7]`)
ALU + execution FSM (config-selected fn)	**replaced by gatling as the execution FSM driver** — the cell's bf16 ALU datapath remains; gatling issues it via POP/PUSH and the alu-infer elastic valid/ready seam (owner ruling 2026-10-19)
MonBus observation (per-cell → per-tetrad merge)	stands; gatling adds per-context status (active/stall-reason/done) to the observed set

Consequences that must not fork: credit counting is in *elements consumed* (POP), not instructions; grants are in *elements pushed* (PUSH), not flits — flit batching stays inside the packetizer.

15.6 6. Verification

- **Base ISA:** spike lockstep on RV32I exactly as kestrel does it — reuse the rds-dv flow; rv32ui battery is the gate (42/42 precedent).
- **Xcell unit:** directed cocotb — POP empty-stall / credit batch N=1,2,4; PUSH grant/backpressure; CFG read of mode shadow; ECTX done/observability; element-width zero-extend and slice.
- **Barrel properties:** cocotb assertions — one retirement per context per rotation window; stall of context k never stalls j≠k; rotation fairness (round-robin, no starvation with all-runnable).
- **Tetrad integration:** the eventual `tetrad.sv` TB drives four input-memory streams, checks per-context result memories and the credit/grant ledger against the cell PRD reference model.
- **Formal (where it pays):** deadlock freedom of the interlock (POP-empty ∨ PUSH-no-grant ∧ no ECTX ⇒ eventually credit ∨ grant), following the repo's targeted-formal doctrine.

15.7 7. Open items (owner decisions needed)

1. **ALU-infer relationship:** RESOLVED 2026-10-19 — gatling is the programmable sequencer for the tetrad's four bf16 ALUs; each context runs a small program whose POP presents operands to the cell's alu-infer issue handshake and whose PUSH retires a result line; the ALU keeps its elastic valid/ready seam and tag-alignment contract unchanged. The v0.1 wording that made gatling "the ALU
   - execution FSM" is superseded.
2. **Context visibility:** RESOLVED 2026-10-21 (owner) — implicit. Software cannot tell the contexts apart; binaries are uniform. CFG[2] stays reserved.
3. **Program image placement:** RESOLVED 2026-10-21 (owner) — separate. Each context has its own program/data region (gatling_mem_bram: one 16 KB bank per context, seen at every address); the cell's POP input memory is separate and arrives with Xcell.
4. **Illegal-instruction policy:** RESOLVED 2026-10-21 (owner) — context-lock. Any halt (ecall, ebreak, illegal, misaligned target or access) locks only its context; the other three keep running.
5. **Xcell binutils/toolchain:** hand-assembled hex for bring-up; gcc/binutils fragment later. Non-blocking.

Resolved/added in v0.2: the FLUSH/KICK/WAIT Xcell encodings of §3.1 (answer TILE_SPEC §11.14 and the alu-infer PRD §5 tetrad-coordination pair). Their exact field layouts and MonBus integration remain provisional pending the tetrad MAS.

Scope of the first RTL (owner, 2026-10-21): the RV32I barrel first – four harts running the rv32ui battery on the Nexys A7 – with the Xcell custom-0 opcode decoding as illegal (context-lock) until the cell seam exists. Xcell lands in the next pass.

15.8 8. Deliverables & sequencing

1. `docs/barrel_model/` study chapter (kestrel cost → interleave fix) — writes first, it is the justification.
2. `gatling_has` + ISA appendix (Xcell table as authority).
3. RTL: fubs (kestrel-derived) → `gatling_top` barrel + Xcell unit → flists; unit TBs per fub on repo rails.
4. Spike lockstep bring-up of the RV32I subset; then Xcell directeds.
5. Tetrad integration when `tetrad.sv` lands in `tile/`.

15.9 9. History

- 2026-10-21: PRD 0.4. The deliberate departures from the RISC-V spec recorded (3.3): mhartid 0, the CSR stub, no trap vector, WFI illegal, a strict rotation. Zicntr counters were added and then removed the same day (owner): unused by any program, ~344 LUTs.
- 2026-10-21: PRD 0.3. First RTL (gatling_core, gatling_regfile, gatling_mem_bram): the RV32I barrel with kestrel's decode/imm_gen/ALU unchanged, every memory a one-stage block-RAM latency, no wait signal; four harts on the Nexys A7. Owner decisions: PRD 7 items 2-4 (implicit context, separate program regions, context-lock); RV32I barrel before Xcell. 4.3 corrected: a taken branch costs no bubble.
- 2026-10-19: PRD 0.2. Owner ruling: gatling sequences the four ALUs of its adjacent cells — the barrel core is the programmable sequencer for the tetrad, the cell's bf16 ALU remains the datapath, and gatling never originates config traffic. Added FLUSH/KICK/WAIT Xcell encodings; fixed POP to read `rd`; updated CFG[0] to report the alu-infer element mode.
- 2026-10-17: PRD 0.1 scaffold. Owner decisions: instructions over MMIO; mini RV per tetrad; codename gatling; four contexts = four cells; blocking POP/PUSH interlock; ECTX exit. Cell PRD 0.1 contracts confirmed unchanged outside the replaced ALU/FSM block.

16 Microarchitecture Reading List

The research literature behind the falcon suite's design decisions. Each entry names the decision it guides and the rung(s) it serves; cite these in rung books and RTL comments the same way the ISA manual is cited (by section/idea, not page). Reading order follows the ladder — foundations first, OOO last; the barrel (gatling) sits beside the ladder, so its sources come after.

Availability is noted honestly: most IEEE/ACM papers are paywalled, but nearly all have free author or institutional mirrors worth finding.

16.1 Foundations — kestrel & merlin

1. **D. Patterson and C. Séquin, "RISC I: A Reduced Instruction Set VLSI Computer," ISCA 1981.** *Guides:* kestrel's whole shape — the case for a register-file-centric datapath, single-cycle simplicity, and why the ISA should map 1:1 to hardware. The original "see everything" machine.

2. **M. Flynn, "Very High-Speed Computing Systems," Proceedings of the IEEE, 1966.** *Guides:* pipelining as a concept (the Flynn taxonomy ride-along); merlin's raison d'être — overlapping instruction stages for throughput rather than raw clock.

3. **J. Hennessy, N. Jouppi, et al., "MIPS: A Microprocessor Architecture," MICRO 1982.** *Guides:* merlin's stage partition (IF/ID/EX/MEM/WB) — the paper that established the five-stage RISC pipeline everyone still teaches.

16.2 Prediction, traps, caches — peregrine

4. **J. E. Smith, "A Study of Branch Prediction Strategies," ISCA 1981.** *Guides:* peregrine's dynamic prediction choice — the original 1-bit counter and its aliasing/measurement methodology; the baseline our 2-bit bimodal improves on.

5. **J. K. F. Lee and A. J. Smith, "Branch Prediction Strategies and Branch Target Buffer Design," IEEE Computer, 1984.** *Guides:* the BTB half of peregrine's frontend — sizing, organization, and the interaction between target caching and direction prediction.

6. **J. E. Smith and A. R. Pleszkun, "Implementing Precise Interrupts in Pipelined Processors," IEEE Transactions on Computers 37(5), May 1988, 562–573.** *Guides:* peregrine's precise-exception strategy — the classic taxonomy (freeze/history-buffer/future-file/reorder-buffer); the reason a simple in-order pipeline can present architecturally exact trap state.

7. **N. Jouppi, "Cache Write Policies and Performance," ISCA 1993.** *Guides:* peregrine's write-back-with-dirty-bit D$ decision — the write-through escape hatch in the spec exists because this paper quantifies when each policy wins.

16.3 Out-of-order — gyrfalcon

8. **R. Tomasulo, "An Efficient Algorithm for Exploiting Multiple Arithmetic Units," IBM Journal of Research and Development 11(1), 1967, 25–33.** (freely available from IBM) *Guides:* gyrfalcon's reservation stations and common data bus — the original OOO. Note what it *lacks* (precise exceptions), which sets up the next two papers.

9. **W. Hwu and Y. Patt, "Checkpoint Repair for High-Performance Out-of-Order Execution Machines," IEEE Transactions on Computers 36(12), 1987, 1496–1514.** *Guides:* gyrfalcon's chosen mispredict recovery — checkpoint the rename state at each branch dispatch and restore on flush, instead of undoing history entry by entry. Our default rollback mechanism.

10. **V. Popescu, M. Schultz, J. Spracklen, et al., "The Metaflow Architecture," IEEE Micro, 1991.** *Guides:* speculative OOO with register renaming as a complete machine — the conceptual ancestor of every P6-style design, and the cleanest statement of "dispatch speculatively, commit in order."

11. **K. Yeager, "The MIPS R10000 Superscalar Microprocessor," IEEE Micro 16(2), April 1996, 28–41.** (free mirrors widely available) *Guides:* gyrfalcon's explicit rename map + active-list organization and branch checkpointing — the closest single reference for our rung 4 design. Also the acknowledged inspiration for BOOM.

12. **R. Kessler, "The Alpha 21264 Microprocessor," IEEE Micro, March–April 1999.** *Guides:* an industrial-strength OOO RISC — clustered scheduling, the register file port problem, and what happens when gyrfalcon's teaching simplifications are removed. Read last.

13. **J. E. Smith and G. Sohi, "The Microarchitecture of Superscalar Processors," Proceedings of the IEEE 83(12), December 1995.** *Guides:* the survey that ties 8–12 together — use it as gyrfalcon's chapter skeleton and as the checklist that no OOO concept was silently dropped.

16.4 ISA rationale & prior art — all rungs

14. **A. Waterman, “Design of the RISC-V Instruction Set Architecture,” PhD dissertation, UC Berkeley, 2016.** (free) *Guides:* why RV32I looks the way it does — the ISA-design rationale behind kestrel’s decode table, and the cheapest way to answer “why does the spec do X?” without reading all 909 pages.

15. **C. Celio, “The Berkeley Out-of-Order Machine (BOOM),” UC Berkeley technical report, 2015.** (free) *Guides:* prior art and differentiation — the open-source OOO RISC-V baseline. gyrfalcon is deliberately *not* BOOM: SystemVerilog rather than Chisel, small enough to hold in your head, checkpoints rather than a unified physical register file. Know it so we don’t accidentally reimplement its complexity.

16.5 Interleaved multithreading — gatling

16. **J. E. Thornton, “Parallel Operation in the Control Data 6600,” Proc. AFIPS Fall Joint Computer Conference, Part II, 1964, 33–40.** *Guides:* the barrel itself. The 6600’s ten peripheral processors shared one execution unit in strict rotation, each taking a slot every ten minor cycles, so no peripheral processor ever had two instructions in flight. gatling’s four-context rotation is that organization at RV32I (barrel model §3). Thornton’s 1970 book *Design of a Computer: The Control Data 6600* (Scott, Foresman) is the long form.

17. **B. J. Smith, “A Pipelined, Shared Resource MIMD Computer,” Proc. International Conference on Parallel Processing (ICPP), 1978, 6–8.** *Guides:* the Denelcor HEP: fine-grained interleaving used to hide pipeline latency with no forwarding and no interlocks between a process’s own instructions, because a process could issue only once its previous instruction had left the pipeline. gatling’s “one instruction in flight per context” invariant (MAS 1.1) is the same rule, fixed by construction instead of by a queue.

18. **H. F. Jordan, “Performance Measurements on HEP — A Pipelined MIMD Computer,” Proc. 10th International Symposium on Computer Architecture (ISCA), 1983, 207–212.** *Guides:* the measurement side of the trade: single-process speed is the cost of the interleave, and throughput arrives only as the process count fills the pipeline. Barrel model §5 and §6 make the same argument with kestrel’s and gatling’s board numbers.

19. **G. Alverson, R. Alverson, D. Callahan, B. Koblenz, A. Porterfield and B. Smith, “Exploiting Heterogeneous Parallelism on a Multithreaded Multiprocessor,” Proc. Supercomputing ’92, 1992, 188–197.** *Guides:* the Tera MTA, the HEP idea taken to 128 streams per processor to cover memory latency without caches. Where gatling stops (four contexts, one block-RAM latency per stage) is read against where Tera went.

20. **P. Kongetira, K. Aingaran and K. Olukotun, “Niagara: A 32-Way Multithreaded Sparc Processor,” IEEE Micro 25(2), March–April 2005, 21–29.** *Guides:* the modern form: four-thread fine-grained cores with round-robin issue that skips a stalled thread. That skip is the textbook norm and the one thing gatling’s rotation deliberately does not do (PRD 3.3; MAS chapter 4).

16.6 Textbooks

The owner's textbook library (machine-local, one Markdown file per section; `vault/handbook/book-library.md`) covers the same ground at textbook depth. Cite by edition, chapter and section, never by page, with the short key in the second column. The chapters the rung books lean on:

Textbooks cited by the rung books

Book	Key	Where it guides
J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed., Morgan Kaufmann, 2019	H&P6 §N	§3.11 fine-grained, coarse-grained and simultaneous multithreading (gatling); Appendix C pipelining and hazards (merlin; gatling's "why not one stream"); Appendix M.5 the lineage from the 6600 I/O processors through HEP, Tera and Niagara (gatling)
D. A. Patterson and J. L. Hennessy, *Computer Organization and Design, RISC-V Edition*, 2nd ed., Morgan Kaufmann, 2021	P&H §N	§4.8-4.9 data and control hazards (kestrel to merlin); §6.4 hardware multithreading (gatling)
S. L. Harris and D. M. Harris, *Digital Design and Computer Architecture, RISC-V Edition*, Morgan Kaufmann, 2022	H&H §N	ch. 6 the RV32I architecture and ch. 7 the single-cycle and pipelined RISC-V microarchitectures (kestrel, merlin); §7.7.7 multithreading as "replicate only the PC and register file" (gatling)
J. Ledin, *Modern Computer Architecture*	Ledin §N	§14.3 simultaneous multithreading, the

Book	Key	Where it guides
and Organization, 2nd ed., Packt, 2022		contrast case for gatling's interleave; ch. 17 the RISC-V ISA
Z. Zhang, *RISC-V Architecture and DSP Processor Design*, Packt, 2026	Zhang §N	ch. 6 arithmetic units (the ALU rungs); a second RISC-V core walk-through to read kestrel against